全部宝石战斗实现 SPEC

2026-11-09 · **草案，未实施** · 游戏1.1.8 · 34个描述符 / 14个行为家族

1. Requirements（需求）

接入全部宝石的独立摆放/武器镶嵌/其他宿主镶嵌效果，兼容现有物品。经DEFAULT_ITEM_FACTORIES创建，独立宝石沿用布局输入；镶嵌由同侧注册后host.add_gems(gem)。数值、阶段、随机、来源、余数及一次性状态须符合矩阵。

仅战斗/几何，不扩JSON插槽、拆装、嵌套、融合、储物箱或局外发育。验收34种三模式/副本/多槽；引擎边界另记。当前仅SPEC，未改代码/测试/数据，未运行测试/引擎。

2. Design（设计）

2.1 依据、结构与状态

先读docs/backpack_battles_knowledge_base.md。唯一权威是Backpack Battles/当前解包；以下源码路径均相对此目录。版本见Core/Game.gd:3-16。恢复物品表80933字节、MD5 dc977faf19965650e7f462c6c9baeef9与加密头一致；恢复证据见docs/decompiled_godotsteam_startup_fix_spec.md:15,114,123。后文CSV为明文物理行，索引为游戏itemIndex。

现有模块	最小变更
item_factory_registry_v1.py	Gem(BattleItem)共享模式、power、额外开战采样和停CD；14家族；五级以partial绑定，仍用普通工厂字典
battle_module_v1.py	状态字段/变更入口、字符串方法监听、眩晕、共享当前疲惫值、最小狂怒/截止时间
item_triggers_v1.py	当前周期阈值、Gem采样、眩晕暂停、最终速度clamp
item_effects_v1.py	现有Effect委托状态入口，读取实例耐力/暴击参数
data/item_catalog_v1.json	只补34种几何/已有元信息；战斗参数留家族常量表

Gem继承集中共有逻辑，用普通字段/同步调用，不加新框架或调度器。保留槽位字段、槽内先行、显式可选validate_gem_sockets，不改initializer/布局函数。函数清单见后文，新增接口先补SPEC。

模式：未镶嵌为I（Inventory）；否则宿主类型含Weapon为W，其余为A（Armor，不要求Armor标签）。p=max(0,gem_power)，默认1；**仅矩阵明确标p的A效果直接乘p。**D=AttackEvent.pending_damage是护盾吸收前、暴击/减伤后的damage；R=godot_round复用battle_module_v1.py:50-51。

状态归属	新增字段（各实例独立）
BattleItem	crit_chance:float=0; crit_multiplier:float=2; stamina_factor:float=1; buff_amplification_chances:dict[str,float]={}; starts_battle_rage:bool=False；已有damage_multiplier复用
Gem	tier:int; gem_power:float=1; cooldown_finished:bool=False及等级参数；家族计数/锁存/截止时间见构造表
PlayerState	float=1: stamina_regeneration,temporary_max_health_gain；float=0: stun_resistance,stunned_until,battle_rage_until; debuff_resist_chances:dict[str,float]={}; debuff_resist_stacks_by_name:dict[str,int]={}; debuff_resist_stacks:int=0; stun_resistance_rng:BalancedRngState; auto_rage_source:BattleItem|None=None; auto_rage_start_time:float|None=None
BattleState / CooldownTrigger	current_fatigue_damage:int=0; iteration_cooldown:float=基础CD，已有progress和speed_modifiers保留
StackChangeContext(BattleContext)	stack_name:str; amount:int; source_item:BattleItem|None; log_handle:LogHandle|None；amount为实际有符号变化，source为本次直接origin

2.2 完整范围与五色数值

五色等级固定chipped/flawed/regular/flawless/perfect，规范ID为{等级}_{家族小写}，下列数组同序，共25种；不能省略regular前缀。

类 / ID后缀	itemIndex	CSV行	脚本依据（Items/Gems/）
Ruby / ruby	86–90	165–169	Ruby.gd:5-18
Sapphire / sapphire	91–95	170–174	Sapphire.gd:10-46
Topaz / topaz	96–100	180–184	Topaz.gd:5-13
Emerald / emerald	101–105	175–179	Emerald.gd:5-19

类 / ID后缀	itemIndex	CSV行	脚本依据（Items/Gems/）
Amethyst / amethyst	106–110	185–189	Amethyst.gd:8-21

全部socket_count=0；shape_cells=[[0,0]]；仅Wisp星位[[-1,-1],[0,-1],[1,-1],[-1,0],[1,0],[-1,1],[0,1],[1,1]]，其他影响格空；筛Nature、实例去重（Items/Exclusive/Wisp.gd:19-23；Wisp.tscn:1752）。item_types：Burning=("Gem","Fire")，Skull/Crystal=("Gem","Dark")，Wisp=("Gem","Nature")，其余=("Gem",)。职业：25五色/Burning/四Rune=()，Lump/Crystal/Skull=("Neutral",)，Wisp=("Ranger",)（Sheets/ItemBook.gd:873-915）；其他已有元信息按恢复表转录。

记号：attacked=宿主完整攻击返回后的signal；early/late=宿主pre_deal_damage对应signal；hit=命中而非生命受伤；mini仅原动画，**不发activated**。一次性=停CD后consume；周期=效果后activate、继续CD。所有计数按实际正变化累计、整除触发且保留余数。

家族	I 独立摆放	W 武器镶嵌	A 其他宿主镶嵌
Ruby	CD5，一次性：Effect伤害[4,6,10,15,35]，按结果D×1.5治疗	attacked且hit：治疗ceil(D×[7,10,15,20,35]/100)，mini	prepare：己治疗效率+=p×[10,15,20,25,40]/100
Sapphire	CD[4.5,4.5,4,3.5,3]，一次性：敌Cold+[2,3,4,5,8]	late roll[15,25,40,60,100]%，成功本次spectral；attacked且本颗spectral、hit：Mana+1、敌Cold+1，mini	每5点己方实际正Mana给Block p×[2,3,4,5,8]，mini；错位参数见2.4
Topaz	prepare：每秒体力恢复+=[.08,.12,.20,.30,.50]，无CD	prepare：宿主speedScale+=[.10,.15,.20,.25,.40]	prepare：眩晕抵抗+=p×[10,15,20,30,50]、暴击抵抗+=p×[5,7,10,15,30]百分点
Emerald	CD[3,4,4,3.5,3]，一次性：Regen+[1,2,3,4,7]	attacked且hit，roll[35,50,80,80,80]%：敌Poison+[1,1,1,2,4]，mini	prepare：Poison抵抗+=p×[10,15,20,25,40]百分点
Amethyst	CD[3.2,2.5,2,1.6,1]，周期：随机净化Debuff1	attacked且hit，roll[25,35,50,75,70]%：移除敌随机Buff[1,1,1,1,2]，mini	prepare：敌治疗效率-=p×[12,18,25,30,45]/100

Ruby I伤害flags=CanBeBlocked|CanTriggerItems|CanCrit（Core/Game.gd:2139-2140）；D×1.5先经Item.heal(amount:int)转换再由角色治疗效率round；W先ceil。Block给层保留float至R(amount*buffPowers[type])，本批buffPowers=1，不能先int截断（Item.gd:4919-4943）。Wisp/Elephant临时最大生命按R(amount*temporary_max_health_gain)同时加max/current，不走治疗效率、不发healed（Character.gd:887-901）。

2.3 九种特殊宝石

下表索引后括号为CSV行。脚本：Lump/Crystal/Skull在Items/Gems/，Burning在Items/，四Rune/Wisp在Items/Exclusive/。

类 / 规范ID / 索引（CSV）	I	W	A
LumpOfCoal / lump_of_coal / 191（14）	CD3，一次性：随机Buff+1，再随机敌Debuff+1	early且hit，roll70%：本次D+=1，mini	prepare全局Debuff抵抗层+=R(p)；start给BlockR(8p)，mini
BurningCoal / burning_coal / 207（40）	CD5，一次性：Heat+2、随机净化Debuff3	early且hit，roll12%：D+=6、Heat+1，mini	prepare Cold专用抵抗层+=R(7p)；start请求Block12p后通用round，mini
CorruptedCrystal / corrupted_crystal / 160（53）	CD3.9，周期：敌bonus_fatigue_damage+=1，立即施加当前共享疲惫源，不推进共享值	只听敌damaged；相对生命<.30-.0001且未加时host.damage_multiplier+=.5；否则已加则减回	敌Debuff实际正变化且直接origin为己方Item，每7层给Block6p，mini
Skull / skull / 156（147）	无CD；首次敌damaged且相对生命<=.30-.0001：先锁存，heal50、Empower+5、activate；不consume	attacked且hit，rollChance2=50%：随机偷Buff1，mini	prepare三Debuff抵抗各+=25p、暴击抵抗+=25p百分点
BadgerRune / badger_rune / 187（317）	无CD；prepare所有根物品stamina_factor=max(0,旧-.1)，不含镶嵌Gem	attacked且hit：host.speedScale+=.03，mini	pre_take_damage：hit、CanTriggerItems、isAttack且狂怒中，请求减伤R(7p)，mini；本模式是自动狂怒候选

类 / 规范ID / 索引（CSV）	I	W	A
ElephantRune / elephant_rune / 188（318）	无CD；start临时max/current+50，consume	attacked且hit先roll25%，再检查time>=ready；成功先ready=time+3，再敌stun.5秒，mini；ICD内仍roll、抵抗stun也进入ICD	prepare三Debuff抵抗各+=40p；start真实4秒Timer，到期减回自身贡献
HawkRune / hawk_rune / 189（319）	CD2.4，周期：敌Blind+1	prepare宿主crit_chance+=12百分点、crit_multiplier+=.15	prepare Blind抵抗+=40p
TigerRune / tiger_rune / 190（320）	无CD；prepare己所有物品含镶嵌Gem，七Buff施加放大率各+11百分点	late且hit，roll50%：Vampirism+1，mini	每10层己方实际正Buff给Block5p，无mini/activate；错位参数见2.4
Wisp / wisp / 495（494）	CD12，一次性：prepare自身speedScale+=Nature星位实例数×.1；到期Lucky+10、Regen+10	attacked且hit，累计D，每32点给Spikes1，mini；无CD	**CD5，一次**：当前生命×.35×p增加临时max/current；停CD、consumed=True、mini，**不consume/activate**

依据：`LumpofCoal.gd:8-27; BurningCoal.gd:8-30; CorruptedCrystal.gd:8-57; Skull.gd:5-34; BadgerRune.gd:5-34; ElephantRune.gd:7-49; HawkRune.gd:5-18; TigerRune.gd:5-40; Wisp.gd:7-59`。

2.4 必须保留的执行边界

- **生命周期**：全体prepare/pre/start/post，阶段内槽先行。Gem start含镶嵌/无CD/零CD皆抽；有CD pre也抽；到期先抽下轮、保余量再效果。Wisp A pre12→start5。普通扰动.95–1.05；低段位Opponent/Lobby.975–1.05未覆盖，非Gem保留固定CD（`Gem.gd:274-315; Item.gd:3339-3395,3788-3795,4409-4442`）。
- **攻击顺序**：hit→early监听/宿主钩子→伤害/暴击/减伤→防方pre→late监听/宿主钩子→防方late→Block/生命→character_attacked→damaged→host.onDealtDamage→死亡/尖刺→吸血→host.attacked。native双倍效果不重复signal监听。煤块early加伤参与暴击，不再吃基础武器倍率（`Character.gd:495-653; Item.gd:4114-4152`）。
- **事件**：damaged含miss/零伤/全Block；Crystal只此重查，Skull先锁存。Stack origin为本次给层Item（`CombatEvent.gd:63-64; CombatLog.gd:722-730`）。父event：Ruby/Sapphire/Emerald/Amethyst W、Skull I、Elephant stun显式传；Skull偷取/Burning Heat/Tiger Vamp/Wisp Spikes不补传。
- **错位参数**：Sapphire/Tiger A保留`giveBlock(p*block,event)`：event落canTriggerEffects，triggerEvent为空；本地传SC对象而非log_handle，对象真值/父链待Godot验证。
- **计时**：stun停CD不停体力；速度`x=speed_modifiers+.02*(Heat-Cold), clamp(x>=0?1+x:1/(1-x),.1,10)`。Elephant抵抗截止不吃速度/stun，ready默认0且prepare不重置；自动rage选择见接口（`Character.gd:46-47,312-336,1551-1589; Util.gd:4,148`）。

2.5 状态规则及公共接口（一）

Buff=`[Luck,Regeneration,Vampirism,Spikes,Mana,Empower,Heat]`，Debuff=`[Poison,Blind,Cold]`，均不含Block；Lucky映射Luck。总抵抗=目标抵抗-来源放大（百分点）：正/负逐原请求层roll抵抗/增层，0不roll；普通`random()<=`概率不套Balanced短路。Debuff再按**放大后amount**做默认0%反弹draw，扣专用/全局抵抗层、限上限（Block100000，其余10000；`Buff.gd:23-41,71-196; Util.gd:1010-1014`）。

非零反弹/临时Buff/移除保护未开放；精确随机0引发的默认0%反弹是已知简化。随机给层均匀选类型；净化/移除/偷取从临时非空类型均匀选、减一删空；完整pick后按首遇类型分组施加，偷取按picked数lose再give（`Item.gd:5405-5509`）。仅Item源的实际非零变化发signal。

接口记法：下列类型缩写仅供本文还原签名，不新增代码别名：`BI=BattleItem; PS=PlayerState; BC=BattleContext; AC=AttackReactionContext; SC=StackChangeContext; LH=LogHandle; Seq=Sequence`。表中每个函数名映射至明确模板，即该具体函数的完整接口；没有列出的override不新增。

```
B = (self, context: BC) -> None
A = (self, context: AC) -> None
S = (self, context: SC) -> None
E = (self, trigger_results: list[bool], context: BC) -> None
P = (self, context: BC) -> bool
Q = (self) -> bool
Z = (self) -> None
J = (self, context: BC, base_cooldown: float) -> float
C5 = (self, item_id: str|None=None, item_key: str|None=None,
      player_id: int|None=None, *, tier: int=2)
```

```
C1 = (self, item_id: str|None=None, item_key: str|None=None,
      player_id: int|None=None)
```

BattleItem.__init__完整签名保持:

```
(self, item_id:str|None=None, item_key:str|None=None, player_id:int|None=None,
 is_active_item:bool=False, triggers:Seq[Trigger]|None=None,
 effects:Seq[Effect]|None=None, linked_items:Seq[str]|None=None,
 display_name:str|None=None, item_classes:tuple[str,...]|None=None,
 rarity:str|None=None, cost:int|None=None, item_types:tuple[str,...]|None=None,
 icon_types:tuple[str,...]|None=None)
```

伪码：沿用槽/RNG/trigger/effect绑定初始化；加2.1字段及每实例字典；允许空effects，不加哑效果。

具体函数 / 接口	伪代码
BattleItem.hasCooldown Q	未镶嵌且有CooldownTrigger才True
BattleItem.adjust_cooldown J	返回base_cooldown，非Gem不改扰动
BattleItem.update_timed_effects B	默认pass
BattleItem.isBattleRageItem Q	False
BattleItem.preCombatStart B	槽内先行及结束检查；自身hasCooldown才初始化CD，最后原业务钩子
Gem.__init__(self,item_id:str|None=None,item_key:str|None=None,player_id:int|None=None,*,tier:int=2,triggers:Seq[Trigger]|None=None,effects:Seq[Effect]|None=None)	校验等级/ID，取家族元信息，初始power/finished；显式传is_active_item=是否有CD trigger至BattleItem；不共享Effect/Trigger
Gem.get_mode(self) -> str	按2.1关系返回inventory/weapon/armor
Gem.get_host(self,context:BC) -> BI|None	未绑定None，否则owner.items[belongs_to]
Gem.get_power(self) -> float	max(0,gem_power)
Gem.hasCooldown Q	I、有正CD且未finished
Gem.adjust_cooldown J	始终抽uniform(.95,1.05)，乘base，含base0
Gem.onCombatStart B	无条件采样；有CD则覆盖iteration并progress=0；模式开战效果在其后
Gem.finish_cooldown(self,context:BC,*,consume:bool=True) -> None	finished=True；consume时调用已有consume，否则仅停CD
CooldownTrigger.__init__(self,cooldown_seconds:float)	原初始化，iteration=基础CD
CooldownTrigger.initialize_for_combat B	progress=0；speed只准备一次；iteration=item.adjust_cooldown
CooldownTrigger.update P	无CD/被stun则False；按2.4速度推进；未到期False；到期减旧iteration保余量、先抽下轮再True；每帧一次

AfterTimeTrigger沿用一次锁存；一次性家族仍显式finish/consume。add_gems/get_gems、显式插槽校验、for_item、resolve_attack、heal、make_attack_event、现有攻击阶段和R数学函数均复用，不改接口。

2.6 公共接口（二）：事件、资源、既有Effect

以下方法属于BattleState，类型/模板见2.5；参数名及关键字位置按接口保留。

函数 / 完整接口	伪代码
connect(self,emitter,signalName:str,listener:tuple[BI,int|str]) -> None	沿用emitter key/列表；str是监听Item方法名
emit_signal(self,emitter:BI|PS|None,signalName:str,event_context:BC|None=None) -> None	保留校验/结束检查/for_item；str→getattr(item,name)(context)，int→原activate_item；不自动激活
change_stacks(self,target:PS,stack_name:str,amount:int,*,source_item:BI|None=None,used:bool=False,trigger_log_handle:LH|None=None) -> SC|None	定位字段；gain按2.5抵抗/放大/0%draw/抵抗层/上限；lose按库存，used须足量；写值/日志；实际非0产生直接source的SC，仅Item源发对应signal；无变化None
gain_block(self,target:PS,amount:float|int,*,source_item:BI,can_trigger_effects:object=True,trigger_log_handle:LH|None=None) -> SC|None	change_stacks(Block,R(amount))；有事件且第二参原版真值成立则source发gave_block；第二参对象与第三参父链分开

函数 / 完整接口	伪代码
`stun(self,target:PS,duration:float,*,source_item:BI,trigger_log_handle:LH|None=None) -> bool`	目标stun Balanced检定，被抵抗则记日志并False；否则until=max(旧,time+duration)，记/发stunned并True
`gain_temporary_max_health(self,target:PS,amount:float|int,*,source_item:BI,trigger_log_handle:LH|None=None) -> int`	gain=R(amount×倍率)；<=0返回0；max/current均加，记录，返回gain，不发healed
`apply_current_fatigue(self,target:PS) -> AttackEvent`	当前共享值固定端点、source=None、fatigue flags→resolve加目标bonus；不推进；返回事件
`inflict_fatigue(self,target:PS,*,nightfall:bool) -> int`	按既有calculate_next公式推进共享值一次、同步两侧镜像；apply_current_fatigue(target)，记delta，返回新值
`pick_random_stacks(self,stack_names:Seq[str],count:int,*,target:PS|None=None) -> dict[str,int]`	target空时独立均匀抽类型；否则按2.5临时正库存抽；按首遇类型分组，不改真实库存
`prepareItems` Z	沿用root shuffle/排序/prepare；重置stun RNG；无starts_battle_rage根物品时，从isBattleRageItem候选随机选一个自动源；active列表额外保留hasCooldown的镶嵌Wisp
`start_combat` Z	沿用幂等/全体分阶段；角色开始时有自动源则设time+5截止
`regenerate_stamina` Z	恢复delta×stamina_regeneration，沿用上限/舍入
`_apply_system_triggers` Z	保留系统tick时间；fatigue_due只inflict_fatigue(enemy,nightfall=True)一次，另一侧apply_current_fatigue；依次查rage开始/结束及所有item.update_timed_effects；保留结束检查
`start_battle_rage(self,target:PS,source_item:BI,duration:float) -> None`	until=time+duration，记/发started
`end_battle_rage(self,target:PS) -> None`	清until，记/发ended；到期只执行一次

下列五个函数统一接口`(self,source_item:BI,count:int,*,trigger_log_handle:LH|None=None)->None`，每行均为独立方法：

具体函数	伪代码（先完整pick，再逐组change_stacks）
`BattleState.give_random_buffs`	七Buff→己正增加
`BattleState.inflict_random_debuffs`	三Debuff→敌正增加
`BattleState.remove_random_buffs`	敌Buff库存pick→敌负变化
`BattleState.cleanse_random_debuffs`	己Debuff库存pick→己负变化
`BattleState.steal_random_buffs`	敌库存pick→敌lose再己give，均按picked数

fatigue由每玩家推进改为共享值、玩家字段镜像，旧测试更新。Crystal不推进；原takeFatigueDamage忽略self、用全局源（`Character.gd:1598-1600`）。公式/17与60秒偏差遗留；PreventAttackDamageEffect累计统计差异不影响胜负时保留。

以下均为现有具体Effect，接口P；攻击Effect另列。返回必须是bool，日志字段保留。

具体函数	伪代码
`GainBuffEffect.apply`	己buff正change，return事件存在且amount>0
`InflictDebuffEffect.apply`	敌debuff正change，return实际>0
`CleanseDebuffEffect.apply`	己debuff负change，return实际<0
`BuffRemovalEffect.apply`	敌buff负change，return实际<0
`UseBuffEffect.apply`	不足False；amount0旧成功True；否则used负change，return实际移除==需求
`GainBlockEffect.apply`	gain_block，return实际>0
`RemoveOpponentBlockEffect.apply`	敌Block负change，return实际<0
`GainMaxHealthEffect.apply`	委托临时max入口，return gain>0
`StaminaConsumptionEffect.apply`	cost=基础×max(0,item.stamina_factor)；据此判断/扣除/记日志，返回成功

具体函数	伪代码
`AttackEffect.apply(self,context:BC,attack_type:tuple[str,...]) -> bool`	原伤害/准确率，事件读clamp(item.crit_chance,0,100)及item.crit_multiplier；resolve后return hit

2.7 家族构造与五色/煤块函数

模板见2.5。五色tier0..4默认2、工厂partial绑定；特殊类内部tier0、工厂直接引用。参数取矩阵，一次/周期trigger分别为AfterTimeTrigger/CooldownTrigger；复用现有Effect及状态入口。

具体构造函数 / 接口	伪代码
`Ruby.__init__` C5	一次trigger；保存吸血/伤害参数
`Sapphire.__init__` C5	一次trigger、Cold Effect；spectral=False，mana_counter=0
`Topaz.__init__` C5	triggers/effects空
`Emerald.__init__` C5	一次trigger、Regen Effect；毒量/chance
`Amethyst.__init__` C5	周期trigger；移除量/chance
`LumpOfCoal.__init__` C1	一次trigger
`BurningCoal.__init__` C1	一次trigger、Heat Effect
`CorruptedCrystal.__init__` C1	周期trigger；damage_bonus_applied=False、debuff_counter=0
`Skull.__init__` C1	无CD；activated=False；Heal/Empower Effect
`BadgerRune.__init__` C1	无CD，无哑Effect
`ElephantRune.__init__` C1	无CD；ready_time=0、resistance_end_time=None、resistance_contribution=0
`HawkRune.__init__` C1	周期trigger、Blind Effect
`TigerRune.__init__` C1	无CD、buff_counter=0
`Wisp.__init__` C1	一次trigger(第一CD)、Luck/Regen Effect、damage_counter=0；保存第二CD

构造均调用Gem，有CD显式active=True；onPrepare接字符串handler，run_effect先查模式/CD。`finish(x)`仅缩写`self.finish_cooldown(context,consume=x)`。

具体函数 / 接口	伪代码
`Ruby.onPrepare` B	W接attacked→on_attack；A加己治疗效率
`Ruby.on_attack` A	hit：按矩阵ceil治疗，source=self、parent=attack.log_handle
`Ruby.run_effect` E	I：make Effect伤害→resolve；int(D×1.5)治疗、父为结果日志；finish(True)
`Sapphire.onPrepare` B	W清spectral，接late→pre_attack及attacked→on_attack；A清Mana计数并接己Mana变化
`Sapphire.pre_attack` A	spectral=roll矩阵chance；成功本次can_be_blocked=False
`Sapphire.on_attack` A	spectral且hit：Mana/Cold，source=self、parent=attack日志
`Sapphire.on_mana_changed` S	正amount累计；n=counter//阈值，counter%=阈值；gain_block矩阵量×n，第二参context，第三None
`Sapphire.run_effect` E	I Cold Effect；finish(True)
`Topaz.onPrepare` B	按I/W/A分别加体力恢复/宿主CD speed/两抵抗
`Emerald.onPrepare` B	W接attacked；A加Poison抵抗
`Emerald.on_attack` A	hit且roll：矩阵Poison，parent=attack日志
`Emerald.run_effect` E	I Regen Effect；finish(True)
`Amethyst.onPrepare` B	W接attacked；A减敌治疗效率
`Amethyst.on_attack` A	hit且roll：remove_random_buffs，parent=attack日志
`Amethyst.run_effect` E	I净化；emit_activation，周期继续
`LumpOfCoal.onPrepare` B	W接early→pre_attack；A加全局Debuff抵抗层
`LumpOfCoal.pre_attack` A	hit且roll：pending_damage+=矩阵bonus
`LumpOfCoal.onCombatStart` B	**先super采样**，再A给矩阵Block

具体函数 / 接口	伪代码
LumpOfCoal.run_effect E	I先随机Buff、再敌Debuff；finish(True)
BurningCoal.onPrepare B	W接early；A加Cold专用抵抗层
BurningCoal.pre_attack A	hit且roll：pending_damage+=bonus，Heat正change，无显式parent
BurningCoal.onCombatStart B	先super采样，再A请求float Block，由入口R
BurningCoal.run_effect E	I Heat Effect、随机净化；finish(True)

2.8 水晶、骷髅、符文、Wisp函数

接口/参数见2.5/2.3；source均本颗宝石，给层经统一入口；计数按实际正变化整除/取余。

具体函数 / 接口	伪代码
CorruptedCrystal.onPrepare B	W清applied，接敌damaged；A清counter，接敌三Debuff变化
CorruptedCrystal.on_opponent_damaged A	读敌当前相对生命；low且未加则host.damage_multiplier加矩阵值并置True，非low且已加则减回；不要求health_damage>0，不听healed
CorruptedCrystal.on_opponent_debuffs_changed S	非正amount、source非BI或非己方则return；累计、整除/取余；给矩阵Block×n，parent=context.log_handle
CorruptedCrystal.run_effect E	I先敌bonus+=矩阵值，再apply_current_fatigue(opponent)，emit_activation；不推进共享值
Skull.onPrepare B	I清锁存，接敌damaged；W接host.attacked；A加三Debuff/暴击抵抗
Skull.on_opponent_damaged A	未锁存且阈值满足，先置True再heal/Empower、activate；两效果source=self、parent=damage日志
Skull.on_attack A	hit且roll：steal_random_buffs，无显式parent
BadgerRune.onPrepare B	I遍历belongs_to为空的己实例改stamina_factor；W接attacked；A接owner.pre_take_damage
BadgerRune.isBattleRageItem Q	mode==armor
BadgerRune.on_attack A	hit则host各CD trigger加矩阵speed
BadgerRune.pre_take_damage A	hit、can_activate_item、melee/ranged且time<rage_until，复用PreventAttackDamageEffect(R(7*p))；不要求damage>0
ElephantRune.onPrepare B	W接attacked，不重置ready；A保存并叠加抵抗贡献
ElephantRune.on_attack A	hit后先roll；失败或time<ready退出；先ready=time+ICD，再stun敌，parent=attack日志；被抵抗仍占ICD
ElephantRune.onCombatStart B	先super采样；I增加临时max/current并consume；A设time+抵抗时长截止
ElephantRune.update_timed_effects B	A截止到期减回自身贡献，清截止；只执行一次
HawkRune.onPrepare B	W改宿主crit_chance/multiplier；A加Blind抵抗
HawkRune.run_effect E	I Blind Effect，emit_activation，继续周期
TigerRune.onPrepare B	I遍历己全部实例含镶嵌Gem，加七Buff放大率；W接late；A清counter、接己七Buff变化
TigerRune.pre_attack A	hit且roll：己Vamp正change，无显式parent
TigerRune.on_buffs_changed S	仅正amount累计、整除/取余；矩阵Block×n，第二参context、第三None；无mini/activate
Wisp.hasCooldown Q	未finished且mode为I或A
Wisp.onPrepare B	I按Nature星位去重加自身CD speed；W清D计数、接attacked
Wisp.onCombatStart B	A先选第二CD、I选第一CD，再super无条件采样
Wisp.on_attack A	hit则counter+=D；n=counter//阈值、取余；Spikes正change，无显式parent

具体函数 / 接口	伪代码
`Wisp.run_effect` E	I Luck/Regen Effect，finish(True)；A按到期current×矩阵比例×p增加临时生命，finish(False)、consumed=True，不完整激活

关键流程（每阶段槽内先行；事件同步）：

```
flowchart LR
    prepare --> pre --> start --> post --> combat[攻击/状态事件]
```

3. Tasks（任务拆解）

全部未实施；文档/测试/战斗数据分别存docs/tests/data，同文件一位写者。

编号 / 依赖	独立任务与可验证产物
[ ] T1 / 无	固定34索引/ID/数值/元信息、14脚本及CSV哈希，确认无漏项
[ ] T2 / T1	总线字符串监听、SC、状态/Block/随机层入口及现有Effect委托；验证直接origin、实际量、随机次数
[ ] T3 / T1	iteration/采样/余量/clamp、stun、耐力/暴击/临时生命、共享fatigue入口；明确兼容语义变化
[ ] T4 / T2,T3	Gem基类、空effects、模式/power/start额外采样、Wisp A轮询例外与finish
[ ] T5 / T4	五色25工厂，逐级三模式验收
[ ] T6 / T4	煤块/水晶/骷髅；early、共享源、阈值、origin计数
[ ] T7 / T4	四符文、5秒后4秒rage、独立4秒抵抗、先roll后ICD、Tiger连锁
[ ] T8 / T4	Wisp双CD/临时生命，34目录项、Nature八邻域
[ ] T9 / T5–T8	多槽/副本/双侧集成；受影响模块和现有全套回归
[ ] T10 / T9	Godot3.6/1.1.8轨迹对照以下边界，证据入data、结论入docs

4. Acceptance Criteria / Tests（验收标准）

按矩阵验收全部数值，组合边界如下：

1. **范围**：34工厂/14家族/精确元信息、单格无槽、Nature去重、现有输入与内存镶嵌；实例RNG/字典/计数/锁存/截止贡献独立。
2. **阶段/CD**：34×I/W/A含无Armor标签宿主；槽先行、分阶段一次；固定RNG核对pre/start/到期及无CD/零CD、Wisp12→5、余量、停止/周期/mini与consumed。
3. **战斗**：p负/0/1/小数，选择性乘p；全Block/零伤/暴击下Ruby/Wisp计D；early加伤、临时spectral、Tiger late、动态crit、体力倍率/恢复、最终speed clamp。
4. **层与来源**：正/负总抵抗roll及默认0反弹draw、抵抗层/上限；actual计数，负量/全抵抗不误计，source=None不发signal；随机非空类型/库存/分组、偷取picked量、跨多阈值及余数。
5. **特殊状态**：阈值附近与damaged miss/全Block/零伤，Crystal回血不立即撤增伤、Skull锁存；Crystal不推进疲惫、系统一次推进；Elephant先roll后ICD/抵抗占ICD/独立撤销；stun停CD不停体力；Badger5→9秒、启动器禁自动、多颗一源。
6. **转换/连锁**：12p半值Block R、Ruby整数转换→效率round、临时max/current不走heal；显式父链/错位参数、原生双倍不倍增signal。

后续在`tests/test_all_gems_v1.py`及既有测试补断言；按AGENTS的Python绝对路径运行专项→受影响模块→全套unittest。T10原引擎对照：

- 错位参数对象真值/gave_block/父链；整数除法/half round/Item.heal:int。
- Gem额外draw；Delayed日志队列分组、直接origin/隐式父event。
- 同帧两侧到期、抵抗撤销、rage/stun/CD、致死后attacked/吸血；RNG精确0反弹。

记录静态/Python专项/全套回归/Godot四层；T10未完成只记“原引擎边界待验证”。未覆盖的补偿环境、精确0反弹、17/60秒疲惫及减伤统计差异保留，不宣称完整原版等价。